Backtracking algorithms

1. Functions and data structures

- N is a constant that denotes the number of variables;
- K is a constant that denotes the domain size (for simplicity it is assumed that all domains have the same size);
- *current* is a variable that contains the number of the current variable;
- v is a one-dimensional array of size N that contains the current instantiations of the variables;

- The main function of every algorithm returns the number of the variable which is selected as the backtracking point (in some cases the return value is not used);
- Function *consistent(current)* returns 1 if the current instantiation is consistent with past instantiations (or, in the case of forward checking algorithms, future variables), and 0 otherwise;

- Function *check(i,j)* returns 1 if the consistency check between v[i] and v[j] succeeds, and 0 otherwise;
- Procedure *solution()* processes the solution stored in the *v* array (if only one solution is sought, it also terminates the algorithm);

- Procedure *merge*(S_1,S_2) merges two sets: $S_1 := S_1 \cup S_2$;
- Procedure *empty(S)* empties a set: $S := \emptyset$;
- Procedure *add(x,S)* adds an element to a set: $S := S \cup \{x\}$;
- Procedure *delete(x,S)* deletes an element from a set: $S := S - \{x\}$;
- Function *max(S)* returns the maximal element of set S.

2. Chronological Backtracking (BT)

```
int consistent(current)
int current;
{
    int i;

    for (i = 1; i < current; i++)
        if (check(current,i) == 0)
            return(0);
    return(1);
}
```

```
int BT(current)
int current;
{
    int i;

    if (current > N) {
        solution();
        return(N); }
    for (i = 0; i < K; i++) {
        v[current] = i;
        if (consistent(current))
            BT(current + 1); }
    return(current-1);
}
```

3. Backjumping (BJ)

- *max_check*[*i*] stores the number of the highest variable that was checked against the current instantiation of x_i?

```
int consistent(current)
int current;
{
    int i;

    for (i = 1; i < current; i++)
        if (check(current,i) == 0) {
            max_check[current] = max(max_check[current],i);
            return(0); }
    max_check[current] = current - 1;
    return(1);
}
```

```
int BJ(current)
int current;
{
    int i, jump;

    if (current > N) {
        solution();
        return(N); }
    max_check[current] = 0;
    for (i = 0; i < K; i++) {
        v[current] = i;
        if (consistent(current)) {
            jump = BJ(current + 1);
            if (jump != current)
                return(jump); } }
    return(max_check[current]);
}
```

4. Conflict-Directed Backjumping (CBJ)

- Instead of the simple array *max_check* we have an array of sets *conf_set*. At a dead-end, *max check*[*i*] corresponds to the maximal element of *conf_set*[*i*].

```
int consistent(current)
int current;
{
    int i;

    for (i = 1; i < current; i++)
        if (check(current,i) == 0) {
            add(i,conf_set[current]);
            return(0); }
    return(1);
}
```

```
int CBJ(current)
int current;
{
    int h, i, jump;

    if (current > N) {
        solution();
        return(N); }
    empty(conf_set[current]);
    for (i = 0; i < K; i++) {
        v[current] = i;
        if (consistent(current)) {
            jump = CBJ(current + 1);
            if (jump != current)
                return(jump); } }
    h = max(conf_set[current]);
    merge(conf_set[h], conf_set[current]);
    return(h);
}
```

5. Graph-Based Backjumping (GBJ)

- Function *parents(i)* returns the PARENTS set of x_i - the set of variables connected to x_i that precede it in the instantiation order.

```
int consistent(current)
int current;
{
    int i;

    for (i = 1; i < current; i++) {
        if (check(current,i) == 0)
            return(0); }
    return(1);
}
```

```
int GBJ(current)
int current;
{
    int h, i, jump;

    if (current > N) {
        solution();
        return(N); }
    for (i = 0; i < K; i++) {
        v[current] = i;
        if (consistent(current)) {
            jump = GBJ(current + 1);
            if (jump != current)
                return(jump); } }
    merge(P,parents(current));
    h = max(P);
    delete(h, P);
    return(h);
}
```

6. Backmarking (BM)

- *mbl*[*i*] contains the number of the lowest variable whose instantiation has changed since the variable x_i was last instantiated with a new value.
- *mcl*[*i*][*j*] contains the number of the highest variable that was checked against the *j*-th value in the domain of the variable x_i.
- all entries in both variables are initially set to 1.

```
int consistent(current)
int current;
{
    int i, oldmbl;

    oldmbl = mbl[current];
    if (mcl[current][v[current]] < oldmbl)
        return(0);
    for (i = oldmbl; i < current; i++) {
        mcl[current][v[current]] = i;
        if (check(current,i) == 0)
            return(0); }
    return(1);
}
```

```
int BM(current)
int current;
{
    int h, i;

    if (current > N) {
        solution();
        return(N); }
    for (i = 0; i < K; i++) {
        v[current] = i;
        if (consistent(current))
            BM(current + 1); }
    h = current - 1;
    mbl[current] = h;
    for (i = h+1; i <= N; i++)
        mbl[i] = min(mbl[i],h);
    return(h);
}
```

7. Forward Checking (FC)

- The integer array *domains* is of size N x K. If *domains*[i][j] = t and t>0 it means that the j-th value has been removed from the domain of variable x_i because of the current instantiation of the variable x_t.
- The boolean array *checking* is of size N x N. The entry *checking*[i][j] is set if the current instantiation of variable xi causes removal of some value from the domain of future variable x_j. Otherwise, it is cleared.

```
void restore(i)
int i;
{
    int j, a;

    for (j = i+1; j <= N; j++)
        if (checking[i][j]) {
            checking[i][j] = 0;
            for (a = 0; a < K; a++)
                if (domains[j][a] == i)
                    domains[j][a] = 0; }
}
```

```
int consistent(current)
int current;
{
    int j, a;
    int old = 0, del = 0;

    for (j = current + 1; j <= N; j++) {
        for (a = 0; a < K; a++)
            if (domains[j][a] == 0) {
                old++;
                v[j] = a;
                if (check(current,j) == 0) {
                    domains[j][a] = current;
                    del++; } }
        if (del)
            checking[current][j] = 1;
        if (old - del == 0)

              return(j); }
    return(0);
}
```

```
int FC(current)
int current;
{
    int i, fail;

    if (current > N) {
        solution();
        return(N); }
    for (i = 0; i < K; i++) {
        if (domains[current][i])
            continue;
        v[current] = i;
        fail = consistent(current);
        if (fail == 0)
            FC(current + 1);
        restore(current); }
    return(current-1);
}
```

- Function *consistent(i)* returns the number of the variable annihilated during forward checking. If no variable is annihilated, the function returns *0*.
- Procedure *restore(i)* undoes the changes caused by the instantiation of x_i.

8. Backmarking and Backjumping (BMJ)
Backmarking and Conflict-Directed Backjumping (BM-CBJ)

```
int consistent(z)
int z;
{
  int i, oldmbl;

  oldmbl = mbl[z];
  if (mcl[z][v[z]] < oldmbl) {

    return(0); }
  for (i = oldmbl; i < z; i++) {
    mcl[z][v[z]] = i;
    if (check(z,i) == 0) {
      max_check[z] =
          max(max_check[z],i);
      return(0); } }
  max_check[z] = z - 1;
  return(1);
}
```

```
=  int consistent(z)
=  int z;
=  {
=    int i, olmbl;
=
=    oldmbl = mbl[z];
=    if (mcl[z][v[z]] < oldmbl) {
#      add(mcl[z][v[z]],conf_set[z]);
=      return(0); }
=    for (i = oldmbl; i < z; i++) {
=      mcl[z][v[z]] = i;
=      if (check(z,i) == 0) {
#        add(i,conf_set[z]);
#
=        return(0); } }
#
=    return(1);
=  }
```

```
int BMJ(z)
int z;
{
  int h, i, jump;

  if (z > N) {
    solution();
    return(N); }
  max_check[z] = 0;
  for (i = 0; i < K; i++) {
    v[z] = i;
    if (consistent(z)) {
      jump = BMJ(z + 1);
      if (jump != z)
        return(jump); } }
  h = max_check[z];

  mbl[z] = h;
  for (i = h+1; i <= N; i++)
    mbl[i] = min(mbl[i],h);
  return(h);
}
```

```
#  int BM_CBJ(z)
=  int z;
=  {
=    int h, i, jump;
=
=    if (z > N) {
=      solution();
=      return(N); }
#    empty(conf_set[z]);
=    for (i = 0; i < K; i++) {
=      v[z] = i;
=      if (consistent(z)) {
=        jump = BM_CBJ(z + 1);
=        if (jump != z)
=          return(jump); } }
#    h = max(conf_set[z]);
#    merge(conf_set[h],conf_set[z]);
=    mbl[z] = h;
=    for (i = h+1; i <= N; i++)
=      mbl[i] = min(mbl[i],h);
=    return(h);
=  }
```

9. Forward Checking and Backjumping (FC-BJ)
Forward Checking and Conflict-Directed Backjumping (FC-CBJ)

```
int FC_BJ(z)
int z;
{
  int h, i, j, jump, fail;

  if (z > N) {
    solution();
    return(N); }
  max_check[z] = 0;
  for (i = 0; i < K; i++) {
    if (domains[z][i])
      continue;
    v[z] = i;
    fail = consistent(z);
    if (fail == 0) {
      max_check[z] = z-1;
      jump = FC_BJ(z + 1);
      if (jump != z)
        return(jump); }
```

```
#  int FC_CBJ(z)
=  int z;
=  {
=    int h, i, j, jump, fail;
=
=    if (z > N) {
=      solution();
=      return(N); }
#    empty(conf_set[i]);
=    for (i = 0; i < K; i++) {
=      if (domains[z][i])
=        continue;
=      v[z] = i;
=      fail = consistent(z);
=      if (fail == 0) {
#
#        jump = FC_CBJ(z + 1);
=        if (jump != z)
=          return(jump); }
```

```
    restore(z);                          =       restore(z);
    if (fail)                            =       if (fail)
      for (j = 1; j < z; j++)            =         for (j = 1; j < z; j++)
        if (checking[j][fail])           =           if (checking[j][fail])
          max_check[z] =                 #             add(j,conf_set[z]); }
          max(max_check[z],j);}          #
  h = max_check[z];                      #
  for (j = 1; j < z; j++)                =     for (j = 1; j < z; j++)
    if (checking[j][z])                  =       if (checking[j][z])
      h = max(h,j);                      #         add(j,conf_set[z]);
                                         #     h = max(conf_set[z]);
                                         #     merge(conf_set[h],conf_set[z]);
  for (i = z; i >= h; i--)               =     for (i = z; i >= h; i--)
    restore(i);                          =       restore(i);
  return(h);                             =     return(h);
}                                        =   }
```

10. BMJ2 – A Modified BMJ

The *mbl* array, which was originally designed for a backstepping algorithm, is no longer adequate for a backjumping algorithm. BM always tests all values of the current variable for consistency. That is why a single entry for all values is sufficient. In BMJ, however, it often happens that only some values of the current instantiation are tested; the other values are skipped by a backjump. A separate entry for each value is necessary to preserve all collected consistency information.

The *mbl*[*i*][*j*] entry stores the number of the lowest variable whose instantiation has changed since the variable x_i was last instantiated with the *j*-th value. The entry is set to 1 in the beginning, and then to *i* every time the current instantiation (x_i, t_j) is being tested for consistency with past instantiations.

```
int consistent(z)                       =  int consistent(z)
int z;                                  =  int z;
{                                       =  {
  int i, olmbl;                         =    int i, oldmbl;
                                        =
  oldmbl = mbl[z];                      #    oldmbl = mbl[z][v[z];
                                        #    mbl[z][v[z]] = z;
  if (mcl[z][v[z]] < oldmbl)            =    if (mcl[z][v[z]] < oldmbl)
    return(0);                          =      return(0);
  for (i = oldmbl; i < z; i++) {        =    for (i = oldmbl; i < z; i++) {
    mcl[z][v[z]] = i;                   =      mcl[z][v[z]] = i;
    if (check(z,i) == 0) {              =      if (check(z,i) == 0) {
      max_check[z] =                    =        max_check[z] =
          max(max_check[z],i);          =            max(max_check[z],i);
      return(0); } }                    =        return(0); } }
  max_check[z] = z - 1;                 =    max_check[z] = z - 1;
  return(1);                            =    return(1);
}                                       =  }
```

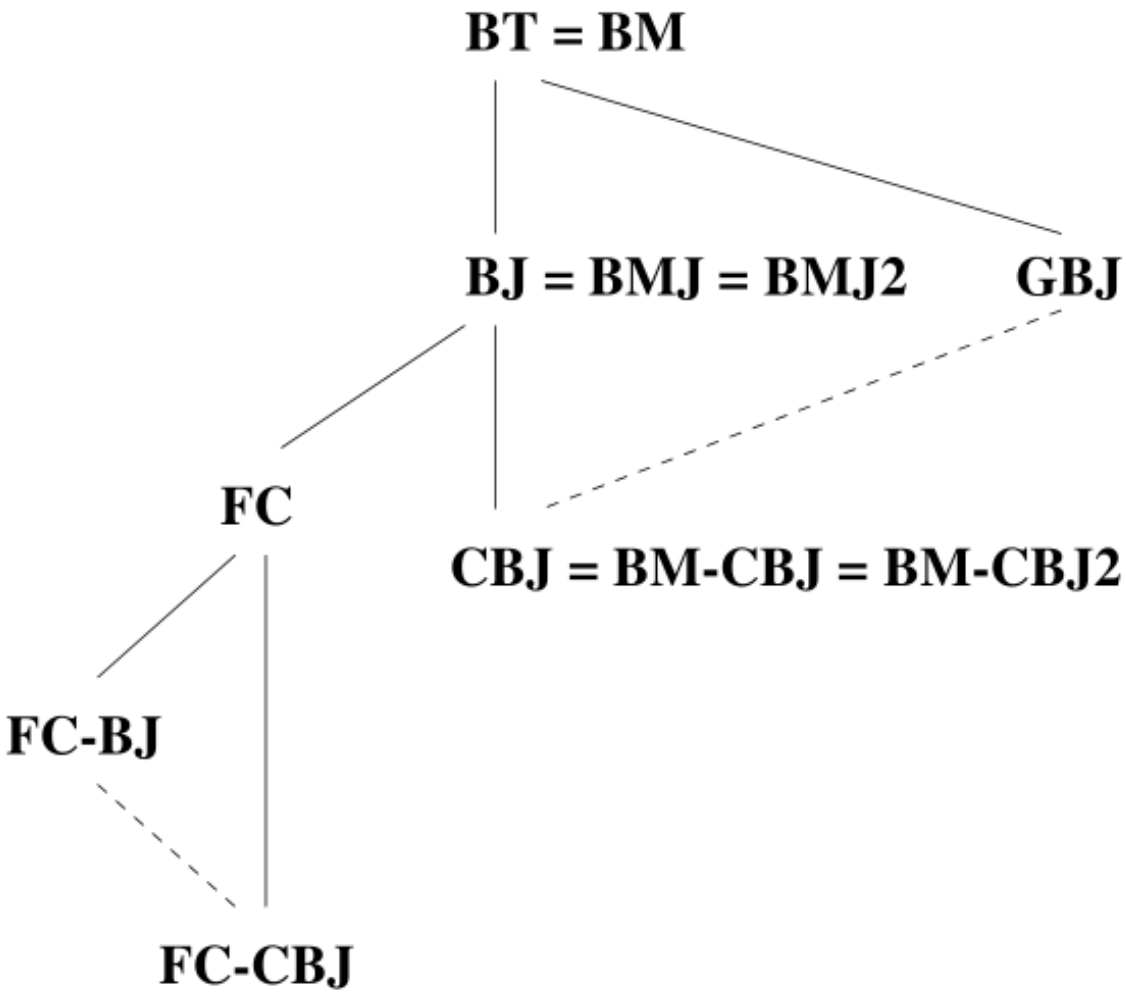
```
int BMJ(z)
int z;
{
  int h, i, jump;

  if (z > N) {
    solution();
    return(N); }
  max_check[z] = 0;
  for (i = 0; i < K; i++) {
    v[z] = i;
    if (consistent(z)) {
      jump = BMJ(z + 1);
      if (jump != z)
        return(jump); } }
  h = max_check[z];
  mbl[z] = h;
  for (i = h+1; i <= N; i++)

    mbl[i] = min(mbl[i],h);
  return(h);
}
```

```
#  int BMJ2(z)
=  int z;
=  {
#    int h, i, j, jump;
=
=    if (z > N) {
=      solution();
=      return(N); }
=    max_check[z] = 0;);
=    for (i = 0; i < K; i++) {
=      v[z] = i;
=      if (consistent(z)) {
=        jump = BM_CBJ(z + 1);
=        if (jump != z)
=          return(jump); } }
=    h = max_check[z];
#
=    for (i = h+1; i <= N; i++)
#      for (j = 0; j < K; j++)
#        mbl[i][j] = min(mbl[i][j],h);
=    return(h);
=  }
```

11. Hierarchy with respect to the number of visited nodes



12. Hierarchy with respect to the number of consistency checks

